Evidence-Guided Kubernetes Upgrade Readiness: A Bounded-Write Operator Artifact with Reproducible Conformance Evaluation

Ihsen Alaya

***Abstract*—Kubernetes upgrade readiness extends beyond deprecated API detection: disruption tolerance, admission behavior, capacity, and remediation evidence must be traceable to cluster state. This paper presents KubeUpgrade Guardian, a bounded-write Kubernetes operator for evidence-guided readiness assessment. It implements `UpgradeAssessment` and `UpgradePlan` with deterministic checkers that produce findings, evidence records, aggregate heuristic priority scores, non-executing decisions, and prioritized remediation actions. The operator reads assessed resources and writes only status and plan artifacts; it never upgrades, drains, or patches assessed workloads. We evaluate on a controlled Kind conformance suite (31 labeled findings, 5 negative controls) and a managed AKS cluster (21 scenario labels, 5 negative controls). In Kind run `20260617T091703Z`, KubeUpgrade Guardian matched all 31 labeled findings with 0 false positives and 0 false negatives; Pluto and kubent matched all 4 deprecated-API findings and are treated as specialized baselines. These results reflect implementation-specification consistency: the evaluation ground truth was authored by the checker team, and should not be interpreted as detection accuracy on independently constructed or adversarial configurations. In AKS run `20260617T101556Z` on Kubernetes `1.34.8`, KubeUpgrade Guardian matched all 21 scenario labels with 0 false positives, 0 false negatives, and 0 negative-control observations; 8 provider-managed webhook observations are reported separately and excluded from scenario metrics. The AKS assessment wait was ˜4 s; peak local controller RSS was ˜47 MiB and the API-server request-counter delta was 11. The replication package additionally includes exploratory scale, ablation, static-baseline, Helm workload, and server-side dry-run artifacts; these are reported as smoke evidence and reproducibility assets rather than calibrated production claims.**

Index Terms

Kubernetes, software evolution, upgrade readiness, cloud-native systems, DevOps, operators, impact analysis, evidence-guided maintenance, empirical software engineering, AIOps.

I. Introduction

Kubernetes has become a common substrate for operating cloud-native software systems, but a production cluster is not merely an execution platform. It is a continuously evolving socio-technical system composed of workloads, controllers, custom resources, admission policies, resource constraints, identity rules, observability pipelines, and managed-provider services. Upgrading such a system is a software evolution problem: a platform version change can alter the validity, behavior, or operational assumptions of dependent artifacts [1], [3], [2]. The practical challenge is not only whether the target Kubernetes version exists. The challenge is whether the specific cluster can be upgraded without exposing application availability, policy, capacity, compatibility, or observability gaps.

The Kubernetes ecosystem already provides authoritative upgrade primitives and policies. The Kubernetes documentation defines API deprecation, removed APIs, version-skew constraints, API-initiated eviction, PodDisruptionBudget behavior, admission webhooks, ValidatingAdmissionPolicy, custom resources, RBAC, and the operator pattern [18], [19], [20], [21], [22], [23], [25], [28], [26], [29]. Managed platforms such as AKS add provider-specific upgrade options, node-image mechanics, upgrade channels, and day-2 practices [33], [34], [35]. These documents are necessary, but they do not by themselves turn a live cluster into a structured, evidence-backed readiness assessment.

Operational teams often complement official guidance with tools such as Pluto, kube-no-trouble (kubent), Helm API migration documentation, and Helm metadata repair tools [36], [37], [38], [39]. These tools are valuable, especially for API removals, but they focus on a subset of readiness risk. A cluster can have no removed API versions and still fail operationally because a single-replica workload is disrupted, a PodDisruptionBudget blocks node drain, an admission webhook is unavailable, a restricted namespace rejects recreated pods, a node pool lacks one-node-loss headroom, or RBAC prevents the platform team from collecting the evidence needed to make a decision.

This paper argues that Kubernetes upgrade readiness should be treated as evidence-guided impact analysis over live cluster state. Rather than producing only warnings or checklist items, an assessment should generate findings that identify affected resources, observable evidence, severity, operational impact, and a non-executing remediation recommendation. Those findings should be materialized in a Kubernetes-native artifact so that platform engineers can review, audit, store, and compare assessments before upgrade execution.

We instantiate this idea in KubeUpgrade Guardian, a read-only-by-design operator artifact. The current

implementation exposes `UpgradeAssessment` and `UpgradePlan`; findings and evidence are structured fields inside those resources, not standalone CRDs. The artifact implements deterministic checkers for workload availability, missing readiness probes, PodDisruptionBudget risks, admission webhooks, policy risks, deprecated or removed APIs, capacity drain risk, observability gaps, and RBAC assessment gaps. It computes aggregate scores and decisions, then creates a prioritized `UpgradePlan` without executing any disruptive action.

a) Research Questions.: This paper investigates five research questions:

RQ1 — Detection Coverage.
To what extent does KubeUpgrade Guardian detect upgrade-readiness risks across controlled and realistic Kubernetes configurations?

RQ2 — Comparative Value.
How does KubeUpgrade Guardian compare with existing Kubernetes analysis tools across different risk families?

RQ3 — Actionability.
Do platform engineers find the generated upgrade plans useful, correct, and actionable?

RQ4 — Scalability.
How does KubeUpgrade Guardian behave as the number of Kubernetes objects increases?

RQ5 — Managed Kubernetes Portability.
Can the approach be applied to managed Kubernetes environments without requiring destructive privileges?

RQ1 and RQ2 are addressed by the controlled evaluation in Section VIII and by the baseline comparison in Section XII. RQ3, RQ4, and RQ5 are open validation questions documented in Section XIII; the current paper provides partial evidence for RQ5 through the AKS smoke-test validation.

The paper makes four scoped contributions.

1) An evidence-guided model for Kubernetes upgrade readiness assessment that links risk categories to observable evidence and non-executing remediation actions.
2) A bounded-write Kubernetes operator artifact implementing `UpgradeAssessment`, `UpgradePlan`, deterministic checkers, scoring, decisions, and idempotent plan generation.
3) An implementation-fidelity analysis that maps manuscript claims to implemented, locally tested, and benchmark-observed artifact behavior. A key scope boundary: the controlled evaluation ground truth was designed by the same team that implemented the checkers. The 0-FP/0-FN result therefore measures implementation-specification consistency, not generalization to unseen cluster configurations. This paper does not claim the latter.
4) A reproducible evaluation package combining a controlled Kind conformance suite, scoped Pluto/kubent baseline comparison, explicit negative controls, managed AKS validation, resource profiling, and archived raw outputs.

II. Background and Problem Statement

A. Upgrade Readiness as Software Evolution

Software evolution research frames long-lived software as a system that must change while preserving useful behavior [1], [3]. Change impact analysis studies how to identify the artifacts affected by a proposed change before that change is made [2]. Kubernetes upgrades fit this pattern: the changed artifact is the platform version, but the affected artifacts include API objects, workloads, admission controllers, add-ons, RBAC permissions, resource requests, and operational workflows.

DevOps and continuous delivery literature emphasizes fast, reliable, automated change delivery, but also documents adoption barriers and the need for reliable feedback loops [4], [5], [6], [7], [8], [9]. SRE practice similarly emphasizes measurable reliability, production readiness, and operational learning [10], [11]. In a Kubernetes upgrade, readiness assessment is one such feedback loop: it should convert cluster state into evidence that engineers can act on before scheduling disruptive work.

B. Kubernetes-Specific Upgrade Mechanics

Kubernetes upgrades are constrained by API deprecation and removal schedules, component version skew, eviction behavior, PodDisruptionBudgets, admission control, RBAC, and resource scheduling. The Kubernetes deprecation guide documents removed APIs by release, while the deprecation policy explains the compatibility process [18], [19]. Version-skew policy constrains how far cluster components may differ during upgrades [20]. API-initiated eviction, including the behavior used by `kubectl drain`, interacts with PodDisruptionBudgets and termination behavior [21], [22]. Admission webhooks and ValidatingAdmissionPolicy can reject newly created or updated objects during post-upgrade reconciliation [23], [25]. Pod Security Admission and Pod Security Standards add another policy layer that may reject workload templates after platform or policy changes [30].

C. Problem Statement

The problem addressed by this paper is:

Given a Kubernetes cluster and a target Kubernetes minor version, how can platform engineers obtain a repeatable, auditable, evidence-backed readiness assessment that covers more than API deprecations while avoiding unsafe automated remediation?

This paper evaluates the bounded artifact at three levels. First, implementation fidelity is checked by mapping claims to the actual CRDs, controller behavior, deterministic checkers, score computation, and generated plan resources. Second, controlled detection behavior is measured

against version-pinned Kind scenarios with labeled positive findings and explicit negative controls. Third, a minimal managed AKS validation checks Kubernetes-modern scenarios and provider-managed admission observations on a real AKS control plane. The study does not claim production-cluster generalization; it reports reproducible scenario-level and managed-cluster smoke-test evidence for the implemented artifact.

D. Scope and Non-Claims

The empirical scope is intentionally bounded. The Kind benchmark is the only environment used for removed/deprecated API fixtures because modern managed API servers reject APIs that have already been removed. The AKS validation uses Kubernetes `1.34.8`, one `Standard_B2s` node, Azure CNI overlay, workload identity enabled, and no production workloads. It validates the non-API readiness families on a managed control plane and records AKS-managed webhook observations, but it does not evaluate real customer traffic, multi-node surge behavior, private-cluster DNS, autoscaler decisions, Azure Policy, or production add-ons. The generated `UpgradePlan` is inspected for structure and evidence linkage, but this paper does not claim external expert actionability ratings. Expert review remains a required next validation step before claiming human operational usefulness.

III. Kubernetes Upgrade Readiness Risk Families

Upgrade readiness is not a single property. It is a set of risk families that interact during upgrade preparation and execution. Table I summarizes the risk taxonomy used by KubeUpgrade Guardian.

The taxonomy is intentionally broader than deprecated APIs. API-removal tools are necessary baselines, but an upgrade may be blocked by disruption semantics, capacity, admission behavior, or insufficient permissions. Treating RBAC denial as an assessment finding is particularly important: an assessment that cannot inspect critical resource types should not silently produce a green result.

Of the ten families in Table I, six are exercised by the controlled conformance evaluation: API evolution, workload availability, readiness probes, PDB/eviction, admission and policy (scoped), and RBAC/evidence gaps. The remaining four—add-on/operator compatibility, managed-provider constraints, full capacity modeling, and observability quality—are included in the taxonomy to bound the problem space but are not used as accuracy claims in this paper. Experimental results in Sections VIII and IX pertain only to the six measured families.

IV. Evidence-Guided Assessment Model

A. Core Data Model

The model is organized around four concepts.

Evidence.
An observable fact collected from the Kubernetes API, such as a replica count, PDB selector, admission webhook failure policy, removed API version, resource request total, or RBAC denial.

Finding.
A risk or assessment gap backed by one or more evidence records. A finding includes an identifier, type, severity, category, optional Kubernetes resource reference, message, evidence, and recommendation.

Assessment.
A bounded snapshot of readiness for a target Kubernetes version and namespace scope. In the artifact, this is materialized as `UpgradeAssessment`.

Plan.
A non-executing remediation artifact that orders required actions and carries the heuristic priority decision. In the artifact, this is materialized as `UpgradePlan`.

a) Formal notation.: Let R be the set of Kubernetes resource objects in the assessed cluster, V the target minor version (e.g., `1.32`), and $\mathcal{F}$ the set of finding types. A checker c produces a set of findings $F_c(R, V) \subseteq \mathcal{F}$. The aggregate finding set is $F = \bigcup_c F_c(R, V)$. The heuristic priority score is $S = \sum_{f \in F} w(f)$, where $w(\text{Critical}) = 25$, $w(\text{High}) = 10$, $w(\text{Medium}) = 4$, $w(\text{Low}) = 1$, and $w(\text{Info}) = 0$. The non-executing decision Δ is determined by: $\Delta = \textsc{DoNotUpgrade}$ if $\exists f \in F$ with $f.\mathit{severity} = \text{Critical}$; $\Delta = \textsc{ProceedWithCaution}$ if $S > 30$; and $\Delta = \textsc{Proceed}$ otherwise. These thresholds are heuristic and not calibrated against production outcomes; S is a deterministic sort key, not a probability of failure.

B. Scoring and Decisions

The current artifact uses a deterministic finding sort key, not a calibrated risk predictor. The weights (Critical: 25, High: 10, Medium: 4, Low: 1, Info: 0) and decision thresholds were chosen to produce a useful ordering of findings for plan generation; they have not been calibrated against production upgrade outcomes or validated by domain experts. The `DoNotUpgrade` label should be read as *assessment recommends blocking: at least one critical finding requires resolution before upgrade*, not as an autonomous upgrade veto.

The threshold triggering `DoNotUpgrade` on a single critical finding is deliberately conservative: it reflects a design choice to err toward caution rather than a calibrated production outcome. A platform team may override this decision after reviewing the specific finding; the artifact provides the evidence for that review, not an autonomous veto.

The aggregate heuristic priority score is interpreted as low below 11, medium from 11 to 30, high from 31 to 60, and critical above 60. A single critical finding yields a `DoNotUpgrade` decision. A score above 30 yields `ProceedWithCaution`. A high or critical RBAC assessment gap can yield `AssessmentIncomplete` when higher-priority decision rules do not already apply. Otherwise, the decision is `Proceed`. The controlled conformance evaluation measures detection consistency and negative-control

TABLE I
KUBERNETES UPGRADE READINESS RISK FAMILIES.

Risk family	Example failure mode	Observable evidence	Current artifact coverage
API evolution	Object uses an API version removed by the target minor release	Group/version/kind, removed-in release, target version	Implemented static MVP table
Workload availability	Critical workload has fewer than two replicas or runs as a standalone pod	Replica count, owner references, namespace scope	Implemented
Readiness	Recreated pod receives traffic before it is ready or cannot signal readiness	Container readiness probe presence	Implemented
PDB/eviction	PDB prevents voluntary disruption during drain	PDB selector, replica count, minAvailable/maxUnavailable	Implemented
Admission and policy	Webhook, ValidatingAdmissionPolicy, or Pod Security setting rejects recreated objects	Webhook configuration, missing services, restricted labels, pod security fields	Partially implemented
Capacity	Cluster cannot absorb one-node-loss or surge capacity during node upgrade	Node allocatable resources, pod requests, remaining headroom	Implemented conservative check
Observability	Upgrade blockers cannot be observed or validated	Monitoring namespaces, Prometheus CRDs, telemetry hints	Implemented heuristic
RBAC and evidence gaps	Assessment cannot inspect required resources	Forbidden API operations and denied resource types	Implemented as findings
Add-on/operator compatibility	Controller or CRD compatibility with target release is unknown	CRDs, Helm releases, controller images, support matrix	Out of current benchmark
Managed-provider constraints	AKS node-pool, image, channel, or surge settings shape feasible plans	Provider metadata, node pool settings, AKS upgrade options	Limited AKS smoke validation

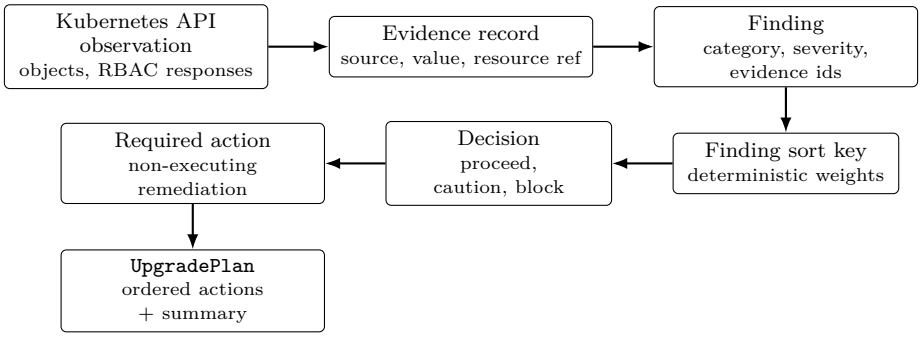

Fig. 1. Evidence pipeline used by the assessment model. Each finding is traceable to collected evidence, and each generated action remains non-executing.

behavior; it does not calibrate severity weights against production outcomes.

A small sensitivity snapshot was computed over the archived Kind and AKS assessment outputs using three scoring variants: current weights, equal non-informational weights, and blocker-first weights. The decision remained `DoNotUpgrade` for both runs because both contain critical findings; a single critical finding forces `DoNotUpgrade` regardless of weight distribution, so this particular result does not stress-test the thresholds. Sensitivity analysis is most informative in the `ProceedWithCaution` range (score 11–60, no critical findings); no such scenario was included in the current evaluation package. This analysis is therefore reported as a consistency check, not as evidence of threshold robustness.

C. Non-Execution Principle

KubeUpgrade Guardian is designed to support readiness assessment, not upgrade execution. The model forbids the assessment tool from upgrading control planes, draining nodes, patching workloads, modifying policies, changing PDBs, or applying remediations automatically. This follows a conservative SRE posture: the assessment should reduce uncertainty before a risky operation, not become another source of unreviewed change [10], [11].

V. KubeUpgrade Guardian Operator

A. Custom Resources

The current artifact implements two CRDs.

- `UpgradeAssessment`: desired assessment target version, mode, namespace scope, selected checks, and observed status including phase, risk level, score, summary, findings, generated plan reference, and conditions.
- `UpgradePlan`: source assessment reference, decision, risk level, score, summary, required non-mutating actions, recommended order, generation timestamp, and conditions.

Findings and evidence are embedded structured fields. They are not standalone CRDs. This is an important fidelity point because a paper that claims separate `UpgradeFinding` or `UpgradeEvidence` resources would not match the implementation.

B. Reconciliation Flow

The controller watches `UpgradeAssessment` resources and owns generated `UpgradePlan` resources. Reconciliation marks the assessment running, executes selected checkers, sorts findings deterministically, computes score and decision, upserts an idempotent plan, and then writes completed assessment status. The controller is a Kubernetes operator in the sense of the Kubernetes operator pattern: it links custom resources to controller behavior without modifying Kubernetes itself [29], [28], [53].

C. Implemented Checkers

Table II maps implemented checkers to evidence and findings.

Admission and policy coverage is deliberately scoped. The evaluated benchmark covers fail-closed webhooks, absent webhook services, broad webhook namespace scope, Pod Security `restricted` enforcement labels, privileged containers, missing security contexts, hostPath volumes, and `allowPrivilegeEscalation=true`. It does not model full admission dry-run semantics, ValidatingAdmissionPolicy expression evaluation, complete Kyverno/Gatekeeper policy semantics, image-policy webhooks, external authorization systems, or provider-managed admission chains.

D. Plan Generation

The planner converts non-informational findings into required actions. Each action includes severity, category, resource reference, remediation text, and evidence references. The default recommended order is: fix critical blockers, fix high-risk workloads, validate admission policies, validate observability, and proceed with controlled upgrade. The order is deterministic and non-executing.

VI. Implementation Fidelity and Artifact Scope

This section maps experimental claims to implemented artifact behavior.

A. Artifact Fidelity

The manuscript was aligned with the operator artifact used by benchmark run `20260617T091703Z` and committed as `b11c531`. The artifact includes:

- Go API types for `UpgradeAssessment`, `UpgradePlan`, `Finding`, `Evidence`, `RequiredAction`, summary, risk levels, and decisions.
- Generated CRD YAML for `UpgradeAssessment` and `UpgradePlan`.
- A controller-runtime reconciler that reads assessed resources, writes assessment status, and creates or updates a generated plan.
- Eight deterministic checker categories and RBAC gap modeling.
- Scoring, decision, and planner packages.
- Helm packaging and GitOps wiring in the repository.

Table III records which claims are supported by the current artifact, local tests, and controlled evaluation outputs.

Features absent from this matrix, including broad field validation, full provider-specific collectors, exact per-controller API attribution, and explanation-agent integration, are not used as empirical claims in this manuscript.

B. Scope Boundaries

The artifact scope is intentionally bounded:

- controlled evaluation and AKS validation metrics are produced by external reproducibility packages, not by the controller implementation itself;
- the removed-API checker uses an MVP static table rather than a complete versioned database;
- AKS validation records cluster metadata and provider webhooks, but not full node-pool upgrade simulation, Azure Policy behavior, or surge-upgrade execution;
- API-server request deltas are cluster-level counters that include harness polling and are not exact per-controller attribution;
- add-on support-matrix reasoning for cert-manager, ingress controllers, policy engines, storage drivers, and observability stacks is outside the core checker set;
- external expert actionability review of `UpgradePlan` is prepared as a protocol but is not claimed as executed;
- explanation-agent behavior is outside the evaluated artifact;
- automatic upgrade, drain, patch, or remediation execution is outside the design.

C. Bounded-Write Semantics

The phrase "read-only" is precise only with respect to assessed cluster resources. The operator does write its own CR status and `UpgradePlan` resources. A stricter description is bounded-write assessment: no assessed workload, node, PDB, policy, webhook, CRD, add-on, or managed-provider resource is mutated by the assessment.

VII. Local Artifact Validation

The local validation objective is artifact sanity, not production effectiveness. The command `go test ./...` was executed against the operator repository and completed with 0 failures. A JSON-formatted run reported 27 passing test events, including subtests, across the checked packages. Packages without tests were reported as such by Go. Table V summarizes the covered behavior.

This test suite supports claims about implemented behavior and deterministic transformations. Controlled recall, false-positive behavior, and baseline coverage are reported separately in Section VIII.

VIII. Controlled Evaluation Results

This section reports the executed empirical evaluation. The conformance suite is intentionally controlled: it tests whether the current artifact detects labeled upgrade-readiness findings in reproducible Kind scenarios and whether safe fixtures remain silent. It does not estimate production-cluster prevalence.

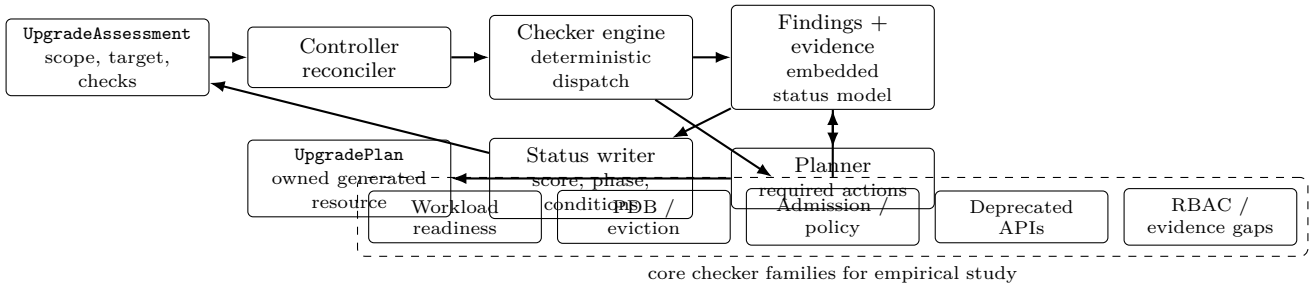

Fig. 2. KubeUpgrade Guardian artifact architecture. The reconciler reads assessment scope, dispatches deterministic checkers, materializes evidence-backed findings, and writes bounded assessment outputs.

TABLE II
Implemented deterministic checkers in the current artifact.

Checker	Triggered finding type	Evidence examples	Artifact status
Workload availability	`WORKLOAD_AVAILABILITY_RISK`	Deployment/StatefulSet replicas below two; standalone pod owner references	Implemented
Readiness probes	`MISSING_READINESS_PROBE`	Container name and missing readiness probe	Implemented
PDB	`PDB_BLOCKING_RISK`	Replica count, selector, `minAvailable`, `maxUnavailable`	Implemented
Admission webhooks	`ADMISSION_WEBHOOK_RISK`	`failurePolicy=Fail`, missing webhook service, broad selector	Implemented
Policy risks	`POLICY_RISK`; `POLICY_ENGINE_DETECTED`	Pod Security restricted labels, privileged containers, hostPath volumes, Kyverno/Gatekeeper CRDs	Partially implemented
Deprecated APIs	`DEPRECATED_OR_REMOVED_API`	Static removed API table, target minor version, live object GVK	MVP table only
Capacity	`CAPACITY_DRAIN_RISK`	Node allocatable CPU/memory, pod requests, one-node-loss headroom	Conservative heuristic
Observability	`OBSERVABILITY_GAP`; `OBSERVABILITY_CAPABILITY_DETECTED`	Monitoring namespace and Prometheus CRD presence	Heuristic
RBAC gap helper	`RBAC_ASSESSMENT_GAP`	Forbidden list/get operations and API resource names	Implemented across checkers

A. Evaluation Setup

The benchmark package is archived under `experiments/kind/r01-benchmark`. The final validated run is `results/20260617T091703Z`. It used Kind image `kindest/node:v1.24.15` to allow creation of Kubernetes APIs removed by later releases, target Kubernetes version `1.32`, Pluto `5.24.0`, and kubent `0.7.3`. KubeUpgrade Guardian version `0.1.1` (commit `b11c531`) was used for both the Kind and AKS runs. The Kubernetes client was `v1.34.1` and the server was `v1.24.15`; this skew is recorded in the tool-version artifact and is acceptable for the benchmark because the run exercises server-side legacy API availability rather than client feature behavior.

The ground truth contains 31 expected findings: 4 deprecated-API findings, 3 workload-availability findings, 5 readiness-probe findings, 7 PDB findings, 4 admission-webhook findings, and 8 selected policy-risk findings. It also contains 5 negative controls: a safe three-replica deployment with readiness and PDB coverage, a scoped fail-open webhook with an existing service, a namespace with warn/audit Pod Security labels but no enforce label, a modern `autoscaling/v2` HPA, and a modern `policy/v1` PDB served by a legacy-capable API server. Raw outputs include the `UpgradeAssessment`, generated `UpgradePlan`, baseline reports, normalized findings, negative-control observations, metrics, controller logs, and tool-version records.

B. Risk Family Occurrence

All six targeted risk families were exercised by the controlled scenarios. The generated assessment reported 31 findings: 7 critical, 13 high, 9 medium, and 2 low. The aggregate heuristic priority score was 343 with a critical heuristic priority level. The generated `UpgradePlan` carried the `DoNotUpgrade` decision, the deterministic recommended order, and 31 non-executing required actions. This result is scenario coverage evidence, not a distribution of risks in real clusters.

C. API-Deprecation Subset

Table VI reports the API-deprecation subset only. KubeUpgrade Guardian, Pluto, and kubent all matched the four deprecated-API labels in both file and cluster modes. This table is the fair comparison for Pluto and kubent because their declared purpose is deprecated Kubernetes API detection.

D. Non-API Readiness Coverage

Table VII maps non-API readiness coverage across tools. KubeUpgrade Guardian covered all 27 non-API labels. Pluto and kubent covered 0/27 because non-API readiness

TABLE III
ARTIFACT FIDELITY MATRIX FOR THE CURRENT MANUSCRIPT CLAIMS.

Feature	Implemented	Locally tested	Used as paper claim	Notes
`UpgradeAssessment` CRD	Yes	Yes	Core artifact claim	Generated CRD and reconciler input.
`UpgradePlan` CRD	Yes	Yes	Core artifact claim	Generated plan resource owned by assessment.
Workload availability checker	Yes	Yes	Core checker claim	Detects low-replica and standalone-pod risks.
Readiness probe checker	Yes	Yes	Core checker claim	Detects missing container readiness probes.
PDB checker	Yes	Yes	Core checker claim	Detects blocking disruption-budget patterns.
Admission webhook checker	Yes	Yes	Core checker claim	Detects risky failure policy, missing services, and broad namespace scope.
Policy risk checker	Partial	Yes	Scoped checker claim	Covers selected Pod Security and policy-engine indicators only.
Deprecated API checker	Partial	Yes	Scoped checker claim	Uses a small static MVP removed-API table and last-applied source filtering to avoid API-server conversion false positives.
Capacity checker	Yes	Yes	Core checker claim	Conservative one-node-loss headroom check.
Observability checker	Yes	Yes	Scoped checker claim	Heuristic signal, not proof of monitoring quality.
RBAC gap modeling	Yes	Yes	Core assessment-gap claim	Forbidden API calls become explicit findings.
Scoring and decision logic	Yes	Yes	Deterministic artifact claim	Heuristic thresholds; not empirically calibrated.
Planner	Yes	Yes	Non-executing plan claim	Produces ordered required actions only.

TABLE IV
CLAIMS AND SUPPORTING EVIDENCE IN THE CURRENT MANUSCRIPT.

Claim	Current evidence	Status	Allowed interpretation
The operator creates `UpgradePlan` resources.	Controller and planner tests; generated CRD; reconciler implementation.	Locally validated	Supported as implemented artifact behavior.
The operator detects low-replica workload scenarios.	Local checker test for low-replica workloads.	Locally validated	Supported for covered fixture behavior.
The operator detects missing readiness probe scenarios.	Local checker test for missing readiness probes.	Locally validated	Supported for covered fixture behavior.
The operator detects a blocking PDB scenario.	Local checker test for a single-replica workload with blocking PDB.	Locally validated	Supported for covered fixture behavior.
The operator detects labeled controlled evaluation findings.	Kind run `20260617T091703Z`: 31 TP, 0 FP, 0 FN against evaluation ground truth.	Controlled evaluation	Supported only for the controlled scenarios and labels in the replication package.
The conformance suite contains explicit negative controls.	Five negative controls produced 0 observations for KubeUpgrade Guardian, Pluto, and kubent.	Controlled evaluation	Supports false-positive checks for the included safe fixtures.
Pluto and kubent cover the API-deprecation subset in the conformance suite.	Pluto/kubent file and cluster runs each matched 4 deprecated-API findings and did not cover the 27 non-API readiness labels.	Controlled evaluation	Supports scoped baseline positioning, not general superiority.
The artifact runs against a managed AKS control plane.	AKS run `20260617T101556Z`: Kubernetes `1.34.8`, 21 scenario TP, 0 FP, 0 FN, 5 negative controls, and 8 provider webhook observations.	Managed validation	Supports managed-cluster smoke-test evidence for Kubernetes-modern non-API scenarios only.
The evaluation records runtime and API-cost proxies.	AKS run `20260617T101556Z`: ~4 s wait, ~47 MiB peak local RSS, 11 cluster-level API-server request-counter delta.	Managed validation	Supports small-scenario profiling only.
The operator models RBAC gaps as findings.	Local checker test for forbidden API access.	Locally validated	Supported for covered RBAC-denial behavior.
The operator performs bounded-write assessment.	Controller implementation and CRD/status write scope.	Implemented behavior	Supports bounded-write artifact description.
The generated plan is externally expert-rated.	No independent platform-engineer rating has been executed.	Not claimed	Requires a future human study before actionability claims.

is outside their declared scope; their purpose is deprecated API discovery, at which they match KubeUpgrade Guardian exactly (Table VI). This table is a functional scope map, not a performance comparison: the baseline tools are not designed to compete on these families.

E. KubeUpgrade Guardian Metrics by Risk Family

Table VIII reports KubeUpgrade Guardian metrics by risk family. The benchmark uses exact resource and message matching for KubeUpgrade Guardian labels. Each expected finding specifies type, severity, resource identity, and a message fragment.

F. Negative Controls

Table IX reports the negative-control observations. All tools produced zero observations for the five safe fixtures. For KubeUpgrade Guardian, this checks the main false-positive concerns in the benchmark: safe multi-replica workloads should not be treated as availability or readiness risks, fail-open scoped webhooks should not be treated as admission blockers, warn/audit-only Pod Security labels should not be treated as enforced policy blockers, and modern API source manifests should not be flagged solely because the API server can serve legacy versions.

TABLE V
LOCAL VALIDATION COVERAGE FROM THE CURRENT GO TEST SUITE.

Area	Covered behavior	Status
Workload availability	Deployment replicas below two are detected with replica evidence	Passed
Readiness probes	Missing container readiness probe is detected	Passed
PDB risk	Single-replica workload with `minAvailable: 1` is critical	Passed
Admission webhook risk	`failurePolicy=Fail` and missing webhook service are detected	Passed
Admission selector scope	Nil and empty namespace selectors are treated as broad scope	Passed
Policy risk	Restricted namespace and privileged workload produce findings	Passed
Deprecated API source filter	Legacy last-applied source is detected while modern source manifests served through legacy endpoints are not flagged	Passed
Capacity	Insufficient one-node-loss headroom is detected	Passed
Observability	Missing monitoring namespace is detected	Passed
RBAC gap	Forbidden list call becomes `RBAC_ASSESSMENT_GAP`	Passed
Controller	Reconciliation creates one idempotent generated plan	Passed
Planner	Required actions and evidence references are generated	Passed
Scoring	Severity weights, risk boundaries, and decision ordering are checked	Passed

TABLE VI
API-DEPRECATION SUBSET METRICS FOR RUN `20260617T091703Z`.

Tool	TP	FP	FN	Precision	Recall	F1
KUBEUPGRADE GUARDIAN	4	0	0	1.0000	1.0000	1.0000
Pluto files	4	0	0	1.0000	1.0000	1.0000
Pluto cluster	4	0	0	1.0000	1.0000	1.0000
kubent files	4	0	0	1.0000	1.0000	1.0000
kubent cluster	4	0	0	1.0000	1.0000	1.0000

TABLE VII
NON-API READINESS COVERAGE FOR RUN `20260617T091703Z`. PLUTO AND KUBENT ARE API-DEPRECATION TOOLS; THEIR 0/27 COVERAGE ON NON-API FAMILIES REFLECTS THEIR DECLARED SCOPE, NOT A DEFICIENCY. THIS TABLE MAPS WHICH RISK FAMILIES EACH TOOL ADDRESSES, NOT WHICH TOOL PERFORMS BETTER.

Tool	Covered	Unexpected	Uncovered	Coverage	F1[a]
KUBEUPGRADE GUARDIAN	27	0	0	1.0000	1.0000
Pluto files	0	0	27	0.0000	0.0000
Pluto cluster	0	0	27	0.0000	0.0000
kubent files	0	0	27	0.0000	0.0000
kubent cluster	0	0	27	0.0000	0.0000

[a]F1 = 0.0000 for Pluto/kubent reflects out-of-scope categories, not detection failure.

TABLE VIII
KUBEUPGRADE GUARDIAN CONTROLLED EVALUATION METRICS BY RISK FAMILY.

Family	TP	FP	FN	F1
Admission webhook	4	0	0	1.0000
Deprecated API	4	0	0	1.0000
PDB	7	0	0	1.0000
Policy risk	8	0	0	1.0000
Readiness probes	5	0	0	1.0000
Workload availability	3	0	0	1.0000

G. Managed AKS Validation

The managed validation package is archived under `experiments/aks/r02-managed-validation`. Run `20260617T101556Z` used AKS cluster `aks-kug-validation-we` in West Europe with Kubernetes `1.34.8`, one `Standard_B2s` system node, Azure CNI overlay, workload identity enabled, and target version `1.35`. The scenario set excludes removed API fixtures because the AKS API server rejects removed API versions before an assessment can observe them. It instead exercises Kubernetes-modern workload availability, readiness, PDB, admission-webhook, and selected policy-risk patterns.

The AKS ground truth contains 21 expected scenario findings and 5 negative controls. KUBEUPGRADE GUARDIAN matched all 21 scenario labels with 0 false positives and 0 false negatives, and produced 0 observations on all 5 negative controls. In addition to scenario findings, KUBEUPGRADE GUARDIAN surfaced 8 AKS-managed admission-webhook observations from provider-managed webhook configurations. These observations are reported separately from scenario-label precision and recall because they are real provider-surface observations, not injected benchmark labels. The 8 provider-managed observations surfaced by KUBEUPGRADE GUARDIAN refer to AKS system admission webhooks installed by the managed control plane (for example, Azure Policy enforcement webhooks and AKS node management webhooks). These are real cluster observations, not injected benchmark labels; they are reported separately so they are not miscounted as scenario false positives.

TABLE IX
NEGATIVE-CONTROL OBSERVATIONS FOR RUN 20260617T091703Z. PF/PC DENOTE PLUTO FILE/CLUSTER MODE; KF/KC DENOTE KUBENT FILE/CLUSTER MODE.

Control	Resource	KUG	PF	PC	KF	KC
Safe deployment	profile-api	0	0	0	0	0
Safe webhook	r01-safe-webhook	0	0	0	0	0
Warn/audit-only policy	r01-policy-warn-audit	0	0	0	0	0
Modern HPA	modern-hpa	0	0	0	0	0
Modern PDB	modern-hpa-target	0	0	0	0	0

TABLE X
KUBEUPGRADE GUARDIAN MANAGED AKS VALIDATION METRICS FOR RUN 20260617T101556Z.

Family	TP	FP	FN	F1
Admission webhook	3	0	0	1.0000
PDB	5	0	0	1.0000
Policy risk	6	0	0	1.0000
Readiness probes	5	0	0	1.0000
Workload availability	2	0	0	1.0000

TABLE XI
MANAGED AKS VALIDATION PROFILE FOR RUN 20260617T101556Z.

Measure	Value
Scoped Deployments	12
Scoped StatefulSets	2
Scoped DaemonSets	1
Scoped PDBs	12
Scoped HPAs	1
Scoped Pods	27
Scenario findings	21
Provider webhook observations	8
Assessment wait	~4 s
Peak local controller RSS	~47 MiB
Cluster-level API request-counter delta	11

H. Execution Cost

The Kind benchmark records wall-clock durations for the assessment wait loop and baseline commands. All timing figures are from a single run on a minimal scenario cluster. They are provided for reproducibility reference only; they carry no statistical confidence and should not be used to compare tool performance. KUBEUPGRADE GUARDIAN completed the assessment wait in ~23 s after scenario application. The KUBEUPGRADE GUARDIAN assessment wait includes operator pod scheduling, leader election, CRD installation, controller warm-up, and reconciliation polling overhead. Pluto and kubent are stateless CLI tools with no operator lifecycle overhead; wall-clock comparison between operator-mode and CLI-mode tools is not meaningful as a performance benchmark. Pluto file mode completed in ~0.2 s and Pluto cluster mode in ~1.6 s. kubent file mode completed in ~0.6 s and kubent cluster mode in ~6.5 s. The AKS validation completed the assessment wait in ~4 s, with ~47 MiB peak local controller RSS. AKS API-server metrics were available; the cluster-level request-counter delta for relevant resources during the assessment window was 11. This counter includes controller requests and harness polling, so it is a cost proxy rather than exact per-controller attribution. These numbers are single-run measurements reported for reproducibility only.

I. Error Analysis

The final Kind run produced 0 KUBEUPGRADE GUARDIAN false positives and 0 KUBEUPGRADE GUARDIAN false negatives against the 31 benchmark labels. The final AKS run produced 0 false positives and 0 false negatives against the 21 AKS scenario labels. Both runs produced 0 observations across their five safe fixtures. The AKS run additionally produced 8 provider-managed webhook observations, which are not treated as scenario false positives because they refer to AKS-managed admission configurations outside the injected ground truth. The main residual risk is benchmark representativeness: the labels cover selected resource patterns and do not exhaust Kubernetes admission, policy, capacity, add-on, or managed-provider behaviors.

Errors are classified into five categories: scenario false positive, scenario false negative, negative-control observation, provider observation outside injected labels, and measurement-attribution ambiguity. The final Kind run has none in the first three categories. The final AKS run has none in the first three categories, 8 provider observations, and one measurement-attribution caveat for the API-server request delta. This taxonomy prevents provider-managed cluster surface from being misreported as benchmark false positives.

J. Interpretation Limits

The controlled conformance evaluation is local, and the AKS validation is a small managed-cluster smoke test. The evaluation still does not include production workloads, multi-node AKS surge upgrades, Azure Policy enforcement, expert actionability ratings, or scoring calibration against outcomes. The result supports scenario-level detection accuracy, negative-control precision for the included safe fixtures, scoped baseline comparison, and limited managed-control-plane feasibility. It does not support claims about production effectiveness, statistically robust accuracy across cluster populations, human-effort reduction, or calibrated risk prediction.

IX. EVALUATION ARTIFACT AND REPRODUCIBILITY

This section describes the produced replication package for the controlled evaluation. The package separates

scenarios, ground truth, tool execution, raw outputs, normalized findings, negative-control observations, and result tables.

A. Produced Artifacts

Table XII lists the files used to reproduce the reported results. The runner creates a temporary Kind cluster, installs the CRDs, starts the controller, applies the scenarios, waits for the assessment, runs Pluto and kubent when available on `PATH`, writes raw outputs, computes normalized metrics, deletes the Kind cluster, and restores the previous kubeconfig context.

B. Replication Command

The controlled benchmark is packaged under `experiments/kind/r01-benchmark`. The run can be reproduced from the repository root with:

```
PATH=/tmp/kug-r01-tools/bin:$PATH \
python3 experiments/kind/r01-benchmark/\
  run_kind_benchmark.py \
  --operator-repo ../kubeupgrade-guardian-operator \
  --restore-context aks-ihsen-mvp-we-admin
```

The runner writes one timestamped directory under `results/`. In the validated run, it restored the kubeconfig context to `aks-ihsen-mvp-we-admin` and deleted the temporary Kind cluster after completion.

C. Managed AKS Validation Package

The managed-cluster validation package is stored separately under `experiments/aks/r02-managed-validation`. It contains AKS-compatible manifests, a ground-truth file with 21 expected non-API findings and 5 negative controls, the runner `run_aks_validation.py`, raw assessment and plan outputs, normalized scenario and provider findings, process samples, scoped object inventory, API-server metric snapshots, and result summaries. The validated run is `results/20260617T101556Z`. It can be reproduced against an existing AKS admin context with:

```
python3 experiments/aks/r02-managed-validation/\
  run_aks_validation.py \
  --context aks-kug-validation-we-admin \
  --resource-group rg-kug-aks-validation-we \
  --cluster-name aks-kug-validation-we
```

The runner deletes its scenario namespaces and webhooks after the run. The validated AKS cluster itself is intentionally not deleted by the runner so that cost and lifecycle control remain explicit.

D. Additional Exploratory Packages

Table XIII summarizes the additional experiment packages now archived in the repository. These packages improve reproducibility and broaden the evidence surface, but they remain exploratory: R02 is a local scale smoke study, R03 and R06 are unlabelled static-baseline runs, R04 is an author-controlled ablation, and R05–R07 validate rendered public Helm corpus handling rather than live add-on operation.

X. Discussion

A. Why Evidence Matters

Evidence changes the review conversation. A finding such as "PDB may block disruption" is more useful when it points to the PDB name, selector, matched workload, replica count, and exact `minAvailable` or `maxUnavailable` value. Similarly, an RBAC-denied list call is not merely a tool error; it is evidence that the assessment cannot establish readiness for a resource family. This makes uncertainty visible.

B. Where the Current Artifact Is Useful

The current artifact is useful as a repeatable pre-upgrade inspection framework, a structured way to expose evidence and assessment gaps, and a basis for comparing broader readiness risk categories against API-deprecation tools. The controlled conformance evaluation shows that the artifact can detect the labeled risks in the current scenario package. Its value is strongest when teams want a Kubernetes-native record of findings and proposed actions without granting the tool authority to mutate workloads.

C. Where the Current Artifact Is Weak

The deprecated-API checker uses a small static table. The observability checker is heuristic. The capacity checker is conservative and does not model all scheduler, autoscaler, topology, taint, priority, or managed-provider behaviors. Policy analysis covers selected indicators and is not a full admission dry run. Add-on and operator compatibility are not part of the measured benchmark. The score is intentionally simple. Until it is calibrated against real upgrade incidents—either through historical incident data or a structured expert study—it should be treated as a ranking aid, not as a risk predictor. Future work should compare score-derived decisions against post-upgrade incident records from real platform teams.

D. Checker Interactions and Plan Dependencies

The current planner treats findings as independently ordered actions, but upgrade readiness risks can compose. A low-replica workload with a strict PDB is more urgent than either finding alone because the same workload lacks redundancy and blocks voluntary disruption. A restricted namespace plus a workload-level security-context finding similarly indicates both policy context and concrete incompatibility. The current artifact preserves these links through shared resource references and evidence fields, but it does not yet compute a dependency graph among actions. In the evaluated plans, critical blockers appear before high-risk workload and admission actions; a future planner should explicitly group actions by affected workload, policy boundary, and disruption path.

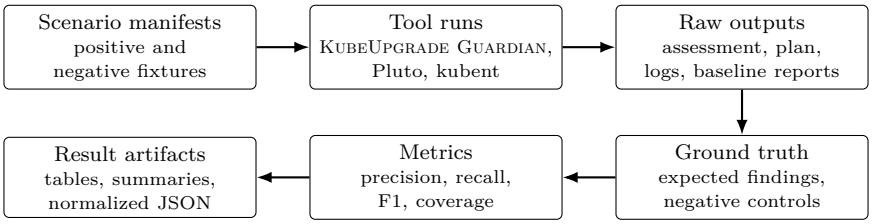

Fig. 3. Controlled evaluation pipeline. The produced package separates scenario inputs, tool execution, raw artifacts, ground truth, metrics, and final result tables.

TABLE XII
Produced evaluation artifacts for run `20260617T091703Z`.

Artifact	Content	Use in results
`manifests/00-scenarios.yaml`	Kubernetes fixtures for positive and negative scenarios	Benchmark input
`ground-truth.json`	31 expected findings and 5 negative controls	Label source
`run_kind_benchmark.py`	Executable benchmark harness	Reproducibility
`upgradeassessment.json`	Raw `UpgradeAssessment` status	Finding counts and risk score
`upgradeplan.json`	Generated `UpgradePlan` with required actions	Decision and remediation output
`normalized-findings.json`	Normalized KubeUpgrade Guardian, Pluto, and kubent findings	Metric computation
`negative-control-observations.json`	Per-control observations by tool	False-positive checks
`metrics.json`	Overall and per-family precision, recall, and F1	Result tables
`summary.md`	Human-readable run summary	Manuscript extraction

TABLE XIII
Additional exploratory experiment packages archived with the replication package.

Package	Scope	Main archived result	Interpretation limit
R02 scale smoke	Synthetic Deployment/PDB pairs at 100, 500, and 1,000 assessed objects; 5 repetitions each.	All 15 assessments completed. Duration mean/std/p95 was 4.556/2.276/7.128 s at 100 objects, 1.642/0.148/1.840 s at 500 objects, and 5.081/2.687/8.755 s at 1,000 objects.	Local smoke only; no 5,000/10,000-object run and no production workload mix.
R03 static baselines	kube-score, kube-linter, and Polaris on the R01 scenario manifest.	JSON parsed for all three tools; primary counts were 185 kube-score failing check instances, 113 kube-linter reports, and 339 Polaris failed check instances.	Feasibility baseline only; no independent TP/FP/FN labels.
R04 ablation smoke	R01 fixtures with one checker family disabled per variant.	Full variant matched 31 TP, 0 FP, 0 FN. Disabling each family introduced only that family's expected false negatives; all variants had 0 negative-control observations.	Author-controlled fixtures; not independent contribution attribution.
R05 Helm corpus	Rendered ingress-nginx, cert-manager, external-dns, kube-prometheus-stack, and Kyverno charts.	281 resources archived, including 38 CRDs; 2 Secrets were excluded from archived manifests.	Rendered corpus only; no live health or upgrade behavior.
R06 Helm static baselines	kube-score, kube-linter, and Polaris over the R05 rendered corpus.	15 JSON-parseable runs. Primary counts totaled 142 kube-score failing check instances, 56 kube-linter reports, and 203 Polaris failed check instances across the five charts.	No independent labels; counts are tool outputs, not accuracy metrics.
R07 Helm server dry-run	Kind `v1.31.0` API-server validation of the R05 corpus.	Applied 38 CRDs and server-side dry-ran 243 non-CRD resources; all five charts succeeded.	API/schema acceptance smoke only; pods are not scheduled and chart webhooks are not made active.

E. CRD Security and Information Exposure

`UpgradeAssessment` status and generated `UpgradePlan` resources can expose operational metadata: workload names, namespaces, selectors, replica counts, policy labels, webhook names, and RBAC-denied resource types. The artifact should therefore be installed with least-privilege RBAC, namespace-scoped access to assessment outputs where possible, and read restrictions on generated plans. A minimal read-only consumer role should grant `get`, `list`, and `watch` on `upgradeassessments` and `upgradeplans` within the installation namespace only. Because ordinary `get`, `list`, and `watch` responses include `status`, environments that need to hide finding details should expose a redacted view rather than relying on the status subresource boundary; at minimum, the consumer role should not grant `update` or `patch` on `status`. The operator's own ClusterRole should be reviewed before installation; in environments where assessment scope is limited to specific namespaces, a namespaced Role with equivalent read permissions can replace the ClusterRole for assessed resources. Findings should avoid secret values, environment variables, ConfigMap payloads, and container arguments that may contain credentials. If used in a shared platform, operators should add redaction rules for

labels or annotations that encode tenant, customer, or incident information. This security boundary follows from the bounded-write design: the artifact avoids mutating assessed resources, but its evidence output still requires access control.

F. Scalability Limits

The AKS validation measured 12 Deployments, 2 StatefulSets, 1 DaemonSet, 12 PDBs, 27 Pods, 1 HPA, and provider webhooks on a one-node cluster. The ˜4 s assessment wait and ˜47 MiB peak local RSS are useful smoke-test numbers, not asymptotic evidence. R02 adds a local repeated scale smoke study over synthetic Deployment/PDB pairs at 100, 500, and 1,000 assessed objects with five repetitions per size. Its p95 assessment durations were 7.128 s, 1.840 s, and 8.755 s respectively, and all 15 assessments completed. These data are a useful first stress signal, but they still do not establish production scalability: the object mix is synthetic, CPU accounting fell below local `/proc` tick resolution, the run omits 5,000 and 10,000 objects, and controller logs contain optimistic-concurrency and storage errors that require separate diagnosis. The current checkers perform straightforward list/get operations and in-memory matching; PDB matching can grow with the number of PDB/workload selector comparisons in a namespace, and admission scanning is cluster-scoped. A stronger scalability study should vary object counts up to at least 10,000 objects, report reconciliation duration, memory, CPU, API-server request deltas, timeout behavior, and separate cache warm-up from assessment execution.

G. Documentation and Review Package

The repository now contains executable benchmark packages and generated summaries, but user-facing documentation still needs a stable installation path, sample `UpgradeAssessment` scopes, RBAC profiles, expected output examples, and cleanup guidance. For human validation, the appropriate artifact is an expert review packet: a generated `UpgradePlan`, source findings, evidence records, and a rubric for correctness, actionability, prioritization, evidence support, and remediation safety. This paper does not report completed external expert ratings; it prepares the structure required to collect them without overstating the current evidence.

H. Cautious Agent Integration

KAGENT is not part of the evaluated artifact [52]. The measured system is deterministic: checkers produce findings, the scorer produces a risk level, and the planner produces non-executing actions. Any agentic explanation layer would need to remain evidence-bounded and non-authoritative; it is not included in the benchmark results.

XI. Threats to Validity

Construct validity. Readiness is multi-dimensional. Precision and recall over labeled findings may not capture business criticality, workload ownership, maintenance-window constraints, or organizational risk tolerance.

Internal validity. Checker defects, incomplete API tables, incorrect benchmark labels, or baseline misconfiguration can bias results. Mitigations include independent scenario review, command archival, deterministic fixtures, and version-pinned tools. Because the benchmark ground truth was designed by the same team that implemented the checker logic, the 0-FP/0-FN result should be interpreted as implementation-specification consistency, not as evidence of detection capability on unseen or adversarially constructed cluster configurations.

External validity. The controlled conformance evaluation demonstrates behavior on labeled scenarios, not production generality. The AKS validation adds managed-control-plane evidence, but it is still a minimal one-node cluster and remains provider-specific. Production clusters may include proprietary controllers, custom admission chains, Azure Policy assignments, private networking, multi-node surge behavior, and hidden operational constraints.

Conclusion validity. The current local test suite supports implementation sanity, the Kind conformance run supports bounded scenario-level accuracy, and the AKS run supports managed-cluster feasibility for Kubernetes-modern non-API scenarios. The data does not support production effectiveness, human-effort reduction, expert-rated actionability, or calibrated risk-prediction claims.

Measurement validity. The AKS API-server request delta is a cluster-level metric over relevant resources during the assessment window. It includes harness polling and cannot be interpreted as exact per-controller request attribution without audit logs or instrumented clients.

Agent validity. The evaluated artifact does not include KAGENT or another explanation agent. Agentic output is therefore outside all reported metrics.

XII. Related Work

A. Software Evolution and Change Impact Analysis

The framing of upgrades as software evolution builds on Lehman's view of long-lived software change and later software evolution work [1], [3]. The evidence-guided model is closest to change impact analysis, where the goal is to identify what a change may affect before applying it [2]. KubeUpgrade Guardian applies this idea to a live Kubernetes cluster by treating the target Kubernetes version as the change and cluster resources as potentially affected artifacts.

B. DevOps, SRE, and Production Readiness

KubeUpgrade Guardian follows the SRE tradition of producing reviewable evidence before risky operations, rather than executing remediations automatically [10], [11]. DevOps and continuous delivery research emphasizes

fast, reliable change delivery and adoption barriers [4], [9]; upgrade readiness assessment is one such feedback loop: it converts cluster state into evidence that engineers can act on before scheduling disruptive work.

C. Infrastructure-as-Code and Kubernetes Configuration Analysis

IaC research shows that infrastructure definitions are software artifacts with defects, security risks, and testing needs [12]. Empirical work on Kubernetes manifests reports configuration and security misconfiguration patterns in real repositories [13], [14]. Recent drift and reconciliation work also frames infrastructure state divergence as a measurable software-engineering problem [51]. These results support the need for assessment tools that reason about concrete configuration and workload state before platform changes.

Static linting tools such as kube-score, kube-linter, and Popeye analyze Kubernetes manifests or live cluster state for best-practice violations [44], [45], [46]. These tools provide useful signal on individual resource quality but do not model cross-resource disruption semantics, target-version upgrade risk, RBAC evidence gaps, or generate a non-executing upgrade plan.

D. Kubernetes API Deprecation and Upgrade Tooling

Kubernetes official documentation defines the authoritative semantics for API removals, version skew, eviction, PDBs, resource requests, and managed upgrade mechanics [18], [19], [20], [21], [22], [31], [33], [34], [35]. Pluto and kubent specialize in deprecated API detection [36], [37]. Helm documentation and mapkubeapis address Helm-specific API migration issues [38], [39]. KubeUpgrade Guardian differs by treating API removals as one risk family among several upgrade-readiness risks. It is not intended to replace Pluto or kubent for exhaustive API-deprecation detection; its contribution is complementary evidence synthesis across non-API readiness families, RBAC evidence gaps, and generated plans. Server-side dry-run (`kubectl apply -dry-run=server`) and admission simulation tools such as `conftest` and `kyverno test` can detect admission rejections for proposed manifests before apply. These tools address one slice of upgrade readiness—admission-time object validity—without synthesizing multi-family readiness findings, evidence records, RBAC gap modeling, or a generated non-executing plan. KubeUpgrade Guardian differs by aggregating across risk families and materializing findings as Kubernetes-native resources.

E. Kubernetes Operators and Operator Reliability

Kubernetes custom resources and the operator pattern define how domain-specific control loops can extend cluster behavior [28], [29], [53]. Empirical operator-bug studies show that operators can fail through incorrect state observation, reconciliation logic, CRD definitions, and access control [15]. This motivates the manuscript's implementation-fidelity emphasis: an operator paper must state precisely which resources it reads, which resources it writes, and which controller behaviors are locally tested.

F. Policy-as-Code and Admission Control

Admission controllers, admission webhooks, ValidatingAdmissionPolicy, Pod Security Admission, and RBAC are central to upgrade readiness because recreated or migrated objects may be rejected even when their API versions still exist [24], [23], [25], [30], [26], [27]. The Kubernetes admission-control threat model further emphasizes that admission components are security-sensitive operational dependencies [32]. Policy-as-code systems such as OPA make policy enforcement programmable, and recent empirical work studies how policy-as-code is adopted in open-source projects [49], [50]. KubeUpgrade Guardian currently performs scoped policy and admission checks; it does not implement a full admission dry run or a complete policy-engine semantics model.

G. AIOps and LLM-Based Operations Assistance

AIOps and LLM assistance can help with explanation and synthesis, but may produce unsupported output and encourage overconfidence [16], [17]. KubeUpgrade Guardian does not include an agentic or LLM layer; any such integration would need to remain evidence-bounded, non-authoritative, and separately evaluated.

XIII. Future Work

The current evaluation establishes implementation fidelity and scenario-level detection consistency. Several directions would strengthen the empirical claims.

a) Independent ground truth.: The controlled evaluation ground truth was authored by the same team as the checker implementation. A credible next step is constructing fixtures and labels independently—by platform engineers who did not write the checkers—and including adversarial scenarios designed to expose false-positive risks.

b) Real-cluster and multi-cluster validation.: Applying KubeUpgrade Guardian to 3–5 real or realistic clusters of different sizes (hundreds to thousands of objects), with genuine add-ons (ingress-nginx, cert-manager, Kyverno, Prometheus stack), multiple Kubernetes target versions, and multi-node node pools would test generalization beyond the current single-node, injected-scenario evaluation. For AKS specifically, future managed-provider validation should include Azure Policy, Cluster Autoscaler behavior, node-image upgrades, multiple node pools, and controlled surge-upgrade execution.

c) Harder benchmark scenarios.: The current fixtures should be extended with ambiguous and near-miss cases: PDB selectors that match no pods or overlap multiple controllers, `maxUnavailable` versus `minAvailable` variants, Deployments with HPAs, StatefulSets, DaemonSets, Jobs and CronJobs, webhook

TABLE XIV
POSITIONING AGAINST ADJACENT KUBERNETES ANALYSIS AND LIFECYCLE TOOLS.

Tool family	Primary purpose	Upgrade-readiness overlap	Boundary relative to KubeUpgrade Guardian
Pluto/kubent	Deprecated API discovery	Strong API-deprecation overlap	Specialized baselines; not general PDB/readiness/policy planners [36], [37].
Helm mapkubeapis	Helm release metadata/API migration	Helm-specific deprecated API migration	Useful for Helm state repair, not live multi-family readiness [39].
`kubectl diff` and `helm diff`	Preview object changes before apply/upgrade	Change preview for manifests or Helm releases	Diffs intended changes; does not synthesize readiness findings or non-executing plans [40], [41].
Datree and Polaris	Static or admission-time policy/configuration checks	Workload best-practice and policy overlap	Broader configuration policy checks; not target-version upgrade impact analysis [42], [43].
kube-score	Workload best-practice scoring	Readiness probe, resource limits, PDB, security context overlap	Scores individual manifests statically; does not model cross-resource disruption paths or produce a Kubernetes-native evidence-backed plan.
Popeye	Live cluster sanitizer	Liveness/readiness probe, unused resources, pod disruption	Scans live clusters; does not link findings to upgrade-target-version risk or generate a non-executing plan.
kube-linter	Static manifest linter	Security context, image tags, resource limits	Static analysis only; no live cluster state, no upgrade-version targeting, no RBAC gap modeling.
kube-bench	CIS Kubernetes Benchmark checks	Cluster security posture context	Security hardening benchmark; not workload disruption or UpgradePlan generation [47].
Cluster API	Declarative cluster lifecycle management	Cluster provisioning/upgrading workflow	Manages cluster lifecycle; KubeUpgrade Guardian assesses readiness without executing upgrades [48].

`namespaceSelector` and `objectSelector` combinations, Pod Security warn/audit/enforce differences, Kyverno and Gatekeeper audit-versus-enforce behavior, modern manifests served through converted APIs, and partial RBAC permissions. These cases would make false-positive and false-negative behavior more informative than the current direct scenario set.

d) Expert human evaluation.: Recruiting platform engineers to rate generated `UpgradePlan` artifacts on correctness, actionability, prioritization quality, evidence usefulness, and remediation safety would provide the human-validation signal currently absent. A structured study with $n \geq 5$ raters and inter-rater agreement measurement is the minimum for a credible actionability claim.

e) Score calibration.: The current severity weights are a deterministic sort key, not a calibrated risk predictor. Calibration against a corpus of real upgrade incidents—matching pre-upgrade assessment scores to post-upgrade outcomes—would allow the score to make evidence-based predictions rather than heuristic orderings.

f) Extended baseline comparison.: The current replication package now includes static kube-score, kube-linter, and Polaris smoke runs on both the R01 scenario manifest and the R05 rendered Helm corpus. These runs demonstrate executable baseline feasibility and expose the kinds of workload-quality findings those tools produce, but they do not yet provide labeled TP/FP/FN comparison. A stronger comparison still requires independent labels over realistic workloads and additional baselines such as Popeye, Datree, Kyverno policy tests, Gatekeeper policy tests, and a manual SRE checklist.

XIV. Conclusion

This paper proposes an evidence-guided impact-analysis approach to Kubernetes upgrade readiness and instantiates it as KubeUpgrade Guardian, a bounded-write Kubernetes-native operator. The approach differs from aggregating specialized CLI tools in three ways: (1) it produces a unified evidence trail linking each finding to its source resource and observable cluster fact; (2) it models assessment uncertainty explicitly through RBAC-gap findings rather than silently omitting inaccessible families; and (3) it materializes findings and remediation plans as persistent, inspectable Kubernetes resources rather than ephemeral command-line output. The current artifact includes deterministic checkers, structured evidence, scoring, decisions, and non-executing remediation actions. Local validation completed with `go test ./...` and 0 failures. The controlled Kind conformance run `20260617T091703Z` reported 31 true positives, 0 false positives, and 0 false negatives for KubeUpgrade Guardian against the labeled scenario package, with 0 observations on 5 negative controls. Pluto and kubent matched all 4 deprecated-API findings in the API subset, while the 27 non-API readiness labels remained outside their scope. The managed AKS validation run `20260617T101556Z` reported 21 true positives, 0 false positives, and 0 false negatives against Kubernetes-modern scenario labels, 0 observations on 5 negative controls, and 8 separately reported AKS-managed webhook observations. Production effectiveness and expert-rated actionability remain open validation questions documented in Section XIII.

References

[1] M. M. Lehman, "Programs, life cycles, and laws of software evolution," *Proceedings of the IEEE*, vol. 68, no. 9, pp. 1060–1076, 1980, doi: 10.1109/PROC.1980.11805.

[2] S. A. Bohner and R. S. Arnold, Eds., *Software Change Impact Analysis*. Los Alamitos, CA, USA: IEEE Computer Society Press, 1996.

[3] T. Mens and S. Demeyer, Eds., *Software Evolution*. Berlin, Heidelberg: Springer, 2008, doi: 10.1007/978-3-540-76440-3.

[4] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.

[5] L. Chen, "Continuous delivery: Huge benefits, but challenges too," *IEEE Software*, vol. 32, no. 2, pp. 50–54, 2015, doi: 10.1109/MS.2015.27.

[6] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017, doi: 10.1109/ACCESS.2017.2685629.

[7] J. Smeds, K. Nybom, and I. Porres, "DevOps: A definition and perceived adoption impediments," in *Agile Processes in Software Engineering and Extreme Programming*, LNBIP 212. Springer, 2015, pp. 166–177, doi: 10.1007/978-3-319-18612-2_14.

[8] L. A. F. Leite, C. Rocha, F. Kon, D. Milojicic, and P. Meirelles, "A survey of DevOps concepts and challenges," *ACM Computing Surveys*, vol. 52, no. 6, 2019, doi: 10.1145/3359981.

[9] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[10] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[11] M. T. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed. Pragmatic Bookshelf, 2018.

[12] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A systematic mapping study of infrastructure as code research," *Information and Software Technology*, vol. 108, pp. 65–77, 2019, doi: 10.1016/j.infsof.2018.12.004.

[13] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 4, article 99, 2023, doi: 10.1145/3579639.

[14] Y. Zhang, U. Paul, M. d'Amorim, and A. Rahman, "Configuration defects in Kubernetes," arXiv:2512.05062, 2025. [Online]. Available: https://arxiv.org/abs/2512.05062

[15] Q. Xu, Y. Gao, and J. Wei, "An empirical study on Kubernetes operator bugs," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1746–1758, doi: 10.1145/3650212.3680396.

[16] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, 2024, doi: 10.1145/3695988.

[17] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" arXiv:2211.03622, 2022. [Online]. Available: https://arxiv.org/abs/2211.03622

[18] Kubernetes Documentation, "Kubernetes deprecation policy." [Online]. Available: https://kubernetes.io/docs/reference/using-api/deprecation-policy/

[19] Kubernetes Documentation, "Deprecated API migration guide." [Online]. Available: https://kubernetes.io/docs/reference/using-api/deprecation-guide/

[20] Kubernetes Documentation, "Version skew policy." [Online]. Available: https://kubernetes.io/releases/version-skew-policy/

[21] Kubernetes Documentation, "API-initiated eviction." [Online]. Available: https://kubernetes.io/docs/concepts/scheduling-eviction/api-eviction/

[22] Kubernetes Documentation, "Disruptions." [Online]. Available: https://kubernetes.io/docs/concepts/workloads/pods/disruptions/

[23] Kubernetes Documentation, "Dynamic admission control." [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/extensible-admission-controllers/

[24] Kubernetes Documentation, "Admission controllers reference." [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/

[25] Kubernetes Documentation, "Validating admission policy." [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/validating-admission-policy/

[26] Kubernetes Documentation, "Using RBAC authorization." [Online]. Available: https://kubernetes.io/docs/reference/access-authn-authz/rbac/

[27] Kubernetes Documentation, "Role based access control good practices." [Online]. Available: https://kubernetes.io/docs/concepts/security/rbac-good-practices/

[28] Kubernetes Documentation, "Custom resources." [Online]. Available: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/

[29] Kubernetes Documentation, "Operator pattern." [Online]. Available: https://kubernetes.io/docs/concepts/extend-kubernetes/operator/

[30] Kubernetes Documentation, "Pod security admission." [Online]. Available: https://kubernetes.io/docs/concepts/security/pod-security-admission/

[31] Kubernetes Documentation, "Resource management for pods and containers." [Online]. Available: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

[32] Kubernetes SIG Security, "Kubernetes admission control threat model." [Online]. Available: https://github.com/kubernetes/sig-security/blob/main/sig-security-docs/papers/admission-control/kubernetes-admission-control-threat-model.md

[33] Microsoft Learn, "Upgrade options and recommendations for Azure Kubernetes Service (AKS) clusters." [Online]. Available: https://learn.microsoft.com/en-us/azure/aks/upgrade-options

[34] Microsoft Learn, "Upgrade Azure Kubernetes Service (AKS) node images." [Online]. Available: https://learn.microsoft.com/en-us/azure/aks/upgrade-node-image

[35] Microsoft Learn, "AKS day-2 guide: Patch and upgrade guidance." [Online]. Available: https://learn.microsoft.com/en-us/azure/architecture/operator-guides/aks/aks-upgrade-practices

[36] FairwindsOps, "Pluto: A CLI tool to help discover deprecated Kubernetes apiVersions," GitHub repository. [Online]. Available: https://github.com/FairwindsOps/pluto

[37] DoiT International, "kube-no-trouble (kubent)," GitHub repository. [Online]. Available: https://github.com/doitintl/kube-no-trouble

[38] Helm Documentation, "Deprecated Kubernetes APIs." [Online]. Available: https://helm.sh/docs/topics/kubernetes_apis/

[39] Helm, "helm-mapkubeapis," GitHub repository. [Online]. Available: https://github.com/helm/helm-mapkubeapis

[40] Kubernetes Documentation, "kubectl diff." [Online]. Available: https://kubernetes.io/docs/reference/kubectl/generated/kubectl_diff/

[41] databus23, "helm-diff: A Helm plugin that shows a diff explaining what a helm upgrade would change," GitHub repository. [Online]. Available: https://github.com/databus23/helm-diff

[42] Datree, "Datree Kubernetes policy CLI," GitHub repository. [Online]. Available: https://github.com/datreeio/datree

[43] FairwindsOps, "Polaris: Open source policy engine for Kubernetes," GitHub repository. [Online]. Available: https://github.com/FairwindsOps/polaris

[44] Zegl, "kube-score: Kubernetes object analysis with recommendations for improved reliability and security," GitHub repository. [Online]. Available: https://github.com/zegl/kube-score

[45] stackrox, "kube-linter: A static analysis tool that checks Kubernetes YAML files and Helm charts," GitHub repository. [Online]. Available: https://github.com/stackrox/kube-linter

[46] derailed, "Popeye: A Kubernetes cluster sanitizer," GitHub repository. [Online]. Available: https://github.com/derailed/popeye

[47] Aqua Security, "kube-bench," GitHub repository. [Online]. Available: https://github.com/aquasecurity/kube-bench

[48] Kubernetes SIG Cluster Lifecycle, "Cluster API." [Online]. Available: https://cluster-api.sigs.k8s.io/

[49] Open Policy Agent, "Open Policy Agent documentation." [Online]. Available: https://www.openpolicyagent.org/docs/

[50] P. L. Foalem, F. Khomh, L. Da Silva, and E. Merlo, "An empirical study of policy-as-code adoption in open-source software

projects," arXiv:2601.05555, 2026. [Online]. Available: https://arxiv.org/abs/2601.05555

[51] Z. Yang, H. Guan, V. Nicolet, B. Paulsen, J. Dodds, D. Kroening, and A. Chen, "Automated cloud infrastructure-as-code reconciliation with AI agents," arXiv:2510.20211, 2025. [Online]. Available: https://arxiv.org/abs/2510.20211

[52] kagent-dev, "kagent: Cloud native agentic AI," GitHub repository. [Online]. Available: https://github.com/kagent-dev/kagent

[53] CNCF TAG App Delivery Operator Working Group, "Operator white paper." [Online]. Available: https://github.com/cncf/tag-app-delivery/blob/main/operator-wg/whitepaper/Operator-WhitePaper_v1-0.md